



decisions.md — Orvio Decision Ledger

Version: 0.1 (MVP-ready)

Last updated: 2026-09-29

Owner: Engineering (Code X)

Applies to: Product, Engineering, Security, DevOps, AI coding agents

0. How to Use This Document

This is the **decision ledger** for Orvio. It records important product, technical, architectural, security, and infrastructure decisions with rationale, alternatives, and consequences.

Chain of truth:

PRD → architecture.md → database.md → api-contract.md → auth.md → security.md → testing.md → roadmap.md → decisions.md → AGENTS.md → Implementation

Rules:

- Do not silently override accepted decisions.
- Do not duplicate existing decisions.
- AI agents must read this before architectural changes.

1. Purpose of decisions.md

Record important decisions that influence design, build, security, test, deploy, or operate.

Answer:

- What was decided?
- Why?
- Alternatives considered?
- Constraints?
- Consequences?
- When to reconsider?

Prevent:

- Repeated debates.
- Contradictory implementations.

- AI agents making arbitrary choices.
- Developers reversing previous decisions.
- Assumptions mistaken for decisions.

2. Source-of-Truth Hierarchy

```
PRD.md
↓
architecture.md
↓
database.md
↓
api-contract.md
↓
auth.md
↓
security.md
↓
testing.md
↓
roadmap.md
↓
decisions.md
↓
AGENTS.md
↓
Implementation
```

Distinction: decisions.md records **why**; other docs record **what/how**. If conflict arises, identify explicitly.

3. Decision Categories

- **Product:** MVP scope, personas, workflows, prioritization.
- **Architecture:** Monolith, frontend/backend, API style, modules.
- **Technology:** Languages, frameworks, runtimes, tools.
- **Database:** Engine, schema, multi-tenancy, IDs, migrations.
- **API:** REST, versioning, errors, pagination, auth, webhooks.
- **Authentication:** Sessions, tokens, cookies, rotation, recovery.
- **Authorization:** RBAC, permissions, tenant isolation.
- **Security:** IDOR, validation, CORS, secrets, audit.
- **Testing:** Pyramid, mocking, CI, coverage.
- **Infrastructure:** Environments, deployment, CI/CD, monitoring.
- **UX:** Navigation, loading, error/empty states, accessibility.

- **Roadmap:** MVP, release, deferred work, sequencing.

4. What Qualifies as a Decision

Record when meaningful consequences for:

- Architecture, product scope, security, data, API compatibility.
- Auth, authZ, infrastructure, testing, workflow, UX, maintainability.

Examples:

- Use React + Vite + TS.
- Use Node.js + Fastify + TS.
- Use PostgreSQL + Prisma.
- JWT access + refresh tokens.
- REST API style.
- Modular monolith initially.

Do not record: Trivial implementation details.

5. Decision Status

- **PROPOSED:** Suggested, not yet accepted.
- **ACCEPTED:** Agreed, active.
- **CONFIRMED:** Established by existing docs/impl.
- **SUPERSEDED:** Replaced by newer decision.
- **DEPRECATED:** Discouraged, phase out.
- **REJECTED:** Considered, not chosen.
- **OPEN:** Not yet decided.

6. Decision Types

- **CONFIRMED DECISION:** Established by project context.
- **PROPOSED DECISION:** Suggested, pending acceptance.
- **OPEN DECISION:** Not yet decided.
- **RECOMMENDATION:** Suggested approach, not binding.
- **ASSUMPTION:** Temporary due to missing info.

Never convert recommendation/assumption to confirmed.

7. Decision Record Format

DEC-001 – <Decision Title>

Status: CONFIRMED

Category: Architecture

Date: UNKNOWN

Source: architecture.md

Context

Why was this decision necessary?

Decision

What was decided?

Alternatives Considered

What alternatives were evaluated?

Why

Why was this option selected?

Consequences

What does this decision make easier/harder?

Constraints

What constraints influenced the decision?

Implementation Impact

What code/architecture/API/infra is affected?

Testing Impact

What tests are required?

Security Impact

What security considerations follow?

Reconsider When

Under what conditions should this be reviewed?

Related Decisions

Links to related DEC-IDs.

8. Decision IDs

Format: DEC-001, DEC-002, ...

- Stable, never reused.
- If superseded, retain old ID and link to new.

9. Decision Timeline

```
Decision
  ↓
Implementation
  ↓
Review
  ↓
Possible Supersession
```

Date: Use UNKNOWN if not documented.

10. Existing Decisions

Extracted from project context:

- **Architecture:** Modular monolith, React + Vite, Node + Fastify, PostgreSQL + Prisma.
- **API:** REST, /api/v1, JWT auth, envelope responses.
- **Auth:** JWT access (15 min) + refresh (7–30 days), httpOnly cookies.
- **AuthZ:** RBAC (Owner, Manager, Cashier, Storekeeper, Accountant), tenant isolation via org_id.
- **Database:** PostgreSQL, UUID IDs, soft delete, Prisma migrations.
- **Security:** Input validation (Zod), HMAC webhooks, rate limiting, secrets in env.
- **Testing:** Vitest (unit/component), Playwright (E2E), API + DB integration.
- **Infra:** PaaS hosting, GitHub Actions CI/CD, Sentry monitoring.
- **Product:** MVP = Inventory + POS + WhatsApp Orders + Payments.

11. Decision Consistency Review

Conflicts identified:

- None confirmed. All docs align on stack, auth, multi-tenancy.

If found:

- Source A vs Source B.
- Current impl.

- Risk.
- Required resolution.

12. Architecture Decisions

DEC-001 — Modular Monolith Architecture

Status: CONFIRMED

Category: Architecture

Source: [architecture.md](#)

Context: Need scalable but simple start.

Decision: Modular monolith (Fastify modules), evolve to services later.

Alternatives:

- Microservices from start (complex, ops burden).
- Pure monolith (harder to split later).

Why: Balance simplicity and future scalability.

Consequences:

- Easier: Deployment, testing, local dev.
- Harder: Extracting services later requires refactoring.

Constraints: Small team, MVP focus.

Implementation Impact: apps/api/src/modules/*.

Testing Impact: Integration tests per module.

Security Impact: Clear boundaries for authZ.

Reconsider When: Scale >10k daily users, team >5 engineers.

Related: DEC-002 (Frontend), DEC-003 (Backend).

DEC-002 — React + Vite + TypeScript Frontend

Status: CONFIRMED

Category: Technology

Source: [architecture.md](#)

Context: Need modern, type-safe frontend.

Decision: React 18 + Vite + TS.

Alternatives:

- Next.js (SSR not needed for MVP).
- Svelte/Vue (team expertise in React).

Why: Team expertise, SPA sufficient, simpler deploy.

Consequences:

- Easier: Hiring, DX, ecosystem.
- Harder: No SSR benefits (SEO not critical).

Constraints: MVP is logged-in app.

Implementation Impact: apps/web/.

Testing Impact: Vitest + RTL.

Security Impact: CSP, XSS via React escaping.

Reconsider When: SEO becomes critical.

DEC-003 — Node.js + Fastify + TypeScript Backend

Status: CONFIRMED

Category: Technology

Source: [architecture.md](#)

Context: Need lightweight, performant API.

Decision: Node.js 20 + Fastify 4 + TS.

Alternatives:

- Express (less structured).
- NestJS (more complex, opinionated).

Why: Team expertise, Fastify balances perf and simplicity.

Consequences:

- Easier: DX, plugin ecosystem.
- Harder: Less opinionated than NestJS.

Constraints: Team skills, MVP speed.

Implementation Impact: apps/api/.

Testing Impact: Vitest for unit/integration.

Security Impact: Input validation, rate limiting plugins.

Reconsider When: Need strong opinions (NestJS) or Go for perf.

13. Technology Decisions

DEC-004 — Prisma ORM for Data Access

Status: CONFIRMED

Category: Technology

Source: [database.md](#)

Context: Need type-safe DB access, migrations.

Decision: Prisma ORM.

Alternatives:

- Kysely (more SQL-like).
- Raw SQL (error-prone).

Why: Type safety, DX, migrations.

Consequences:

- Easier: Schema management, queries.
- Harder: Some runtime overhead, vendor lock-in.

Constraints: TypeScript stack.

Implementation Impact: `apps/api/prisma/`.

Testing Impact: Test DB per CI run.

Security Impact: Parameterized queries (no SQLi).

Reconsider When: Need advanced SQL features Prisma lacks.

14. Database Decisions

DEC-005 — PostgreSQL as Primary Database

Status: CONFIRMED

Category: Database

Source: [database.md](#)

Context: Need relational, transactions, reporting.

Decision: PostgreSQL 15+.

Alternatives:

- MySQL (less features).
- MongoDB (no transactions for complex ops).

Why: Relational data, constraints, reporting.

Consequences:

- Easier: Complex queries, FKs.
- Harder: Operational complexity vs serverless DBs.

Constraints: Multi-tenant, transactional integrity.

Implementation Impact: `apps/api/prisma/schema.prisma`.

Testing Impact: Ephemeral Postgres in CI.

Security Impact: Row-level security future.

Reconsider When: Scale requires sharding (consider CockroachDB).

DEC-006 — UUID Primary Keys

Status: CONFIRMED

Category: Database

Source: [database.md](#)

Context: Need non-guessable IDs.

Decision: UUID for all PKs.

Alternatives:

- Auto-increment integers (guessable).
- ULID/ksuid (less standard).

Why: Security, distributed-friendly.

Consequences:

- Easier: No ID enumeration.
- Harder: Larger indexes, slower joins.

Constraints: Security, multi-tenant.

Implementation Impact: All tables id UUID.

Testing Impact: Factories generate UUIDs.

Security Impact: Prevents ID guessing.

Reconsider When: Performance critical (consider ULID).

DEC-007 — Soft Delete with `deleted_at`

Status: CONFIRMED

Category: Database

Source: [database.md](#)

Context: Need audit, recovery.

Decision: Soft delete (`deleted_at` TIMESTAMPTZ).

Alternatives:

- Hard delete (no recovery).
- Archive tables (complex).

Why: Audit, NDPA compliance.

Consequences:

- Easier: Recovery, audit.
- Harder: Queries must filter `deleted_at` IS NULL.

Constraints: Compliance, audit.

Implementation Impact: All core tables have `deleted_at`.

Testing Impact: Tests verify soft delete.

Security Impact: Data retained for audit.

Reconsider When: Storage cost prohibitive.

DEC-008 — Multi-Tenancy via `org_id` Scoping

Status: CONFIRMED

Category: Database

Source: [database.md](#), [security.md](#)

Context: Need tenant isolation.

Decision: All rows scoped by `org_id`; app-layer enforcement.

Alternatives:

- Separate databases per tenant (expensive).
- Row-Level Security (RLS) (complex for MVP).

Why: Simplicity, cost.

Consequences:

- Easier: Single DB, simple queries.

- Harder:** Must enforce `org_id` in every query.

Constraints: MVP speed, cost.

Implementation Impact: Repository base class injects `org_id`.

Testing Impact: Tenant isolation tests.

Security Impact: Critical (prevent cross-org access).

Reconsider When: Scale requires RLS or separate DBs.

15. API Decisions

DEC-009 — REST API Style

Status: CONFIRMED

Category: API

Source: [api-contract.md](#)

Context: Need simple, cacheable API.

Decision: REST (JSON), versioned `/api/v1`.

Alternatives:

- GraphQL (flexible but complex).
- gRPC (internal, not client-facing).

Why: Simplicity, caching, team expertise.

Consequences:

- Easier:** DX, caching, tooling.
- Harder:** Over/under-fetching.

Constraints: MVP, React client.

Implementation Impact: `apps/api/src/modules/*/routes.ts`.

Testing Impact: Contract tests.

Security Impact: AuthZ per endpoint.

Reconsider When: Need flexible queries (GraphQL).

DEC-010 — Envelope Response Format

Status: CONFIRMED

Category: API

Source: [api-contract.md](#)

Context: Need consistent responses.

Decision: { data, meta, error } envelope.

Alternatives:

- Direct resource (simpler but inconsistent).
- JSON:API (verbose).

Why: Consistency, extensibility.

Consequences:

- Easier:** Pagination, errors.
- Harder:** Slightly more verbose.

Constraints: Frontend expectations.

Implementation Impact: All endpoints return envelope.

Testing Impact: Response schema tests.

Security Impact: Error structure hides internals.

Reconsider When: Adopt JSON:API.

DEC-011 — Cursor-Based Pagination

Status: CONFIRMED

Category: API

Source: [api-contract.md](#)

Context: Need scalable pagination.

Decision: Cursor-based (?cursor=abc&limit=25).

Alternatives:

- Offset pagination (slow for large datasets).
- Page numbers (inconsistent with cursors).

Why: Performance, consistency.

Consequences:

- Easier:** Large datasets.

- Harder:** Client complexity (store cursor).

Constraints: Reporting, large lists.

Implementation Impact: All collection endpoints.

Testing Impact: Pagination tests.

Security Impact: Prevents enumeration attacks.

Reconsider When: Small datasets only.

16. Authentication Decisions

DEC-012 — JWT Access + Refresh Tokens

Status: CONFIRMED

Category: Authentication

Source: [auth.md](#)

Context: Need stateless auth, mobile-friendly.

Decision: JWT access (15 min) + refresh (7-30 days).

Alternatives:

- Sessions only (server state).
- OAuth2 only (complex for MVP).

Why: Stateless, simple, mobile-ready.

Consequences:

- Easier:** Scaling, mobile.
- Harder:** Token revocation strategy.

Constraints: MVP speed.

Implementation Impact: `apps/api/src/modules/auth/`.

Testing Impact: Token expiry, rotation tests.

Security Impact: Short-lived access, refresh rotation.

Reconsider When: Need advanced session mgmt (sessions + JWT hybrid).

DEC-013 — httpOnly Cookies for Refresh Tokens

Status: CONFIRMED

Category: Authentication

Source: [auth.md](#)

Context: Prevent XSS token theft.

Decision: Refresh token in httpOnly cookie.

Alternatives:

- LocalStorage (XSS vulnerable).
- Memory only (no persistence).

Why: Security (XSS protection).

Consequences:

- Easier:** Secure persistence.
- Harder:** CSRF considerations (mitigated by JWT in Authorization header).

Constraints: Browser security.

Implementation Impact: Auth middleware sets cookie.

Testing Impact: Cookie security tests.

Security Impact: Critical (prevent token theft).

Reconsider When: Mobile apps (use secure storage).

17. Authorization Decisions

DEC-014 — RBAC with Tenant Isolation

Status: CONFIRMED

Category: Authorization

Source: [security.md](#), [database.md](#)

Context: Need role-based access, tenant isolation.

Decision: RBAC (Owner, Manager, Cashier, Storekeeper, Accountant) + org_id scoping.

Alternatives:

- ABAC (complex for MVP).
- ACL per resource (verbose).

Why: Simplicity, clarity.

Consequences:

- Easier:** Permission mgmt.
- Harder:** Custom roles future.

Constraints: MVP, team size.

Implementation Impact: Middleware checks role + org_id.

Testing Impact: IDOR, tenant isolation tests.

Security Impact: Critical (prevent cross-org access).

Reconsider When: Need fine-grained permissions (ABAC).

18. Security Decisions

DEC-015 — Input Validation with Zod

Status: CONFIRMED

Category: Security

Source: [security.md](#)

Context: Prevent injection, invalid data.

Decision: Zod schemas at API boundary.

Alternatives:

- Joi (older).
- Yup (less TS-friendly).

Why: TS integration, DX.

Consequences:

- Easier:** Type safety.
- Harder:** Schema duplication (mitigated by shared types).

Constraints: TypeScript stack.

Implementation Impact: All routes validate with Zod.

Testing Impact: Validation error tests.

Security Impact: Prevents injection, ensures data integrity.

Reconsider When: Need runtime type inference (Zod suffices).

DEC-016 — HMAC Signature Verification for Webhooks

Status: CONFIRMED

Category: Security

Source: [security.md](#)

Context: Prevent webhook forgery.

Decision: Verify HMAC signature on Paystack/WhatsApp webhooks.

Alternatives:

- No verification (insecure).
- IP allowlist (less secure).

Why: Security, provider standard.

Consequences:

- Easier:** Trust webhooks.
- Harder:** Key management.

Constraints: Payment security.

Implementation Impact: `apps/api/src/modules/webhooks/`.

Testing Impact: Signature verification tests.

Security Impact: Critical (prevent fraud).

Reconsider When: None.

19. Testing Decisions

DEC-017 — Vitest + Playwright Testing Stack

Status: CONFIRMED

Category: Testing

Source: [testing.md](#)

Context: Need fast, reliable tests.

Decision: Vitest (unit/component), Playwright (E2E).

Alternatives:

- Jest (slower, more config).
- Cypress (slower E2E).

Why: Speed, TS integration.

Consequences:

- Easier:** DX, parallel runs.
- Harder:** Playwright learning curve.

Constraints: TypeScript, CI speed.

Implementation Impact: `apps/*/tests/`.

Testing Impact: Fast CI, reliable E2E.

Security Impact: Security tests in CI.

Reconsider When: Need visual regression (add Percy).

DEC-018 — Ephemeral Test Database per CI Run

Status: CONFIRMED

Category: Testing

Source: [testing.md](#)

Context: Need isolated, reproducible tests.

Decision: Ephemeral Postgres (Testcontainers) per CI run.

Alternatives:

- Shared test DB (flaky).
- In-memory DB (not Postgres-compatible).

Why: Isolation, reliability.

Consequences:

- Easier:** Parallel tests.
- Harder:** CI setup complexity.

Constraints: CI reliability.

Implementation Impact: CI config spins up Postgres.

Testing Impact: No cross-test contamination.

Security Impact: No prod data in tests.

Reconsider When: CI cost prohibitive.

20. Product Decisions

DEC-019 — MVP Scope: Inventory + POS + WhatsApp Orders + Payments

Status: CONFIRMED

Category: Product

Source: [PRD.md](#), [roadmap.md](#)

Context: Focus on core SMB pain points.

Decision: MVP includes Inventory, POS, WhatsApp Orders, Paystack.

Alternatives:

- Broader MVP (purchasing, reports).
- Narrower MVP (inventory only).

Why: Balance value and complexity.

Consequences:

- Easier:** Focused development.
- Harder:** Deferred features (purchasing).

Constraints: Team size, time.

Implementation Impact: Prioritize modules.

Testing Impact: MVP E2E flows.

Security Impact: Secure core workflows first.

Reconsider When: User feedback demands otherwise.

21. UX Decisions

DEC-020 — Skeleton Loading for Dashboards/Lists

Status: CONFIRMED

Category: UX

Source: [testing.md](#), [architecture.md](#)

Context: Improve perceived performance.

Decision: Skeleton loading for dashboards, lists.

Alternatives:

- Spinners (less informative).
- Full-page loaders (blocking).

Why: UX best practice.

Consequences:

- Easier:** User feedback.
- Harder:** Implement skeleton components.

Constraints: Performance.

Implementation Impact: apps/web/src/components/.

Testing Impact: Loading state tests.

Security Impact: None.

Reconsider When: Design system changes.

22. Infrastructure Decisions

DEC-021 — PaaS Hosting (Render/Railway/Fly)

Status: CONFIRMED

Category: Infrastructure

Source: [roadmap.md](#)

Context: Need simple deploy, managed DB.

Decision: PaaS for MVP.

Alternatives:

- VPS (more ops).
- Serverless (cold starts, complexity).

Why: Speed, managed infra.

Consequences:

- Easier:** Deploy, backups.
- Harder:** Less control, cost at scale.

Constraints: Team size, MVP.

Implementation Impact: CI/CD to PaaS.

Testing Impact: Staging/prod envs.

Security Impact: Managed TLS, backups.

Reconsider When: Scale requires Kubernetes.

DEC-022 — GitHub Actions for CI/CD

Status: CONFIRMED

Category: Infrastructure

Source: [roadmap.md](#)

Context: Need reliable CI/CD.

Decision: GitHub Actions.

Alternatives:

- CircleCI, Travis (less integrated).
- Jenkins (complex).

Why: GitHub integration, free tier.

Consequences:

- Easier:** Workflow config.
- Harder:** Vendor lock-in.

Constraints: GitHub repo.

Implementation Impact: `.github/workflows/`.

Testing Impact: CI runs tests on PR.

Security Impact: Secrets masked.

Reconsider When: Need advanced pipelines.

23. Development Workflow Decisions

DEC-023 — Monorepo with pnpm Workspaces

Status: CONFIRMED

Category: Development Workflow

Source: [architecture.md](#)

Context: Share code between web/api.

Decision: Monorepo (apps/web, apps/api, packages/shared).

Alternatives:

- Separate repos (harder sharing).
- Turborepo (more complex).

Why: Code sharing, single source.

Consequences:

- Easier:** Shared types, configs.
- Harder:** Build orchestration.

Constraints: TypeScript, shared types.

Implementation Impact: `pnpm-workspace.yaml`.

Testing Impact: Shared test utils.

Security Impact: None.

Reconsider When: Team scales, split repos.

24. AI Coding Agent Decisions

DEC-024 — AI Agents Must Respect Existing Decisions

Status: CONFIRMED

Category: AI Coding Agent

Source: [AGENTS.md](#), [decisions.md](#)

Context: Prevent AI from making arbitrary choices.

Decision: AI agents must read [decisions.md](#) before architectural changes.

Alternatives:

- No restrictions (risk inconsistency).
- Manual review only (slow).

Why: Preserve architecture.

Consequences:

- Easier:** Consistency.
- Harder:** AI must check docs.

Constraints: AI capabilities.

Implementation Impact: [AGENTS.md](#) rules.

Testing Impact: AI-generated code tested.

Security Impact: Prevent security bypass.

Reconsider When: AI tooling improves.

25. Decision Change Rule

Process:

```
Existing Decision
  ↓
Reason for Reconsideration
  ↓
Impact Analysis
  ↓
Alternatives
  ↓
New Decision
  ↓
Supersede Old Decision
  ↓
Update Affected Documentation
  ↓
Update Implementation
  ↓
Update Tests
```

Do not reverse casually.

26. Superseding Decisions

Example:

```
DEC-004 – Original Decision
Status: SUPERSEDED
Superseded By: DEC-019

DEC-019 – New Decision
Status: CONFIRMED
Supersedes: DEC-004
```

Preserve history.

27. Decision Reversibility

Classify:

- **Easy to Reverse:** UI components.
- **Moderately Expensive:** API endpoints.
- **Expensive:** Database schema.
- **Very Expensive:** Auth architecture, multi-tenancy.

28. Decision Consequences

Every decision documents:

- Benefits (what becomes easier).
- Costs (what becomes harder).

29. Alternatives

For significant decisions:

- Option A, B, C.
- Advantages, disadvantages, compatibility, complexity, security, ops.

30. Decision Drivers

Examples:

- Simplicity, performance, security, expertise, cost, time, scalability, maintainability, vendor independence, product requirements.

If unknown: Rationale: Not documented.

31. Decision Invariants

- Accepted decisions cannot be silently reversed.
- Security decisions cannot be bypassed.
- API contracts cannot be changed casually.
- DB changes follow migrations.
- Auth respects `auth.md`.
- Security respects `security.md`.
- Testing respects `testing.md`.
- AI agents don't create duplicate solutions.

32. Decision Matrix

ID	Decision	Category	Status	Impact	Reversibility
DEC-001	Modular Monolith	Architecture	CONFIRMED	High	Moderate
DEC-002	React + Vite	Technology	CONFIRMED	High	Easy
DEC-003	Node + Fastify	Technology	CONFIRMED	High	Moderate
DEC-004	Prisma ORM	Technology	CONFIRMED	High	Moderate

ID	Decision	Category	Status	Impact	Reversibility
DEC-005	PostgreSQL	Database	CONFIRMED	High	Expensive
DEC-006	UUID PKs	Database	CONFIRMED	Medium	Expensive
DEC-007	Soft Delete	Database	CONFIRMED	Medium	Moderate
DEC-008	Multi-Tenancy via org_id	Database	CONFIRMED	Critical	Very Expensive
DEC-009	REST API	API	CONFIRMED	High	Expensive
DEC-010	Envelope Responses	API	CONFIRMED	Medium	Easy
DEC-011	Cursor Pagination	API	CONFIRMED	Medium	Moderate
DEC-012	JWT Access + Refresh	Auth	CONFIRMED	Critical	Very Expensive
DEC-013	httpOnly Refresh	Auth	CONFIRMED	Critical	Very Expensive
DEC-014	RBAC + Tenant Isolation	AuthZ	CONFIRMED	Critical	Very Expensive
DEC-015	Zod Validation	Security	CONFIRMED	High	Moderate
DEC-016	HMAC Webhooks	Security	CONFIRMED	Critical	Very Expensive
DEC-017	Vitest + Playwright	Testing	CONFIRMED	High	Moderate
DEC-018	Ephemeral Test DB	Testing	CONFIRMED	High	Moderate
DEC-019	MVP Scope	Product	CONFIRMED	Critical	Moderate
DEC-020	Skeleton Loading	UX	CONFIRMED	Medium	Easy
DEC-021	PaaS Hosting	Infra	CONFIRMED	High	Moderate
DEC-022	GitHub Actions CI/CD	Infra	CONFIRMED	High	Moderate
DEC-023	Monorepo with pnpm	Workflow	CONFIRMED	High	Moderate
DEC-024 — AI Agents Respect Decisions	AI	CONFIRMED	High	Moderate	

33. Open Decisions

Question	Why It Matters	Affected Areas	Known Options	Dependencies	Owner
Email provider?	Verification, resets	Auth, Notifications	SendGrid, Resend, AWS SES	NDPA compliance	OPEN
OAuth providers?	Social login	Auth	Google, Microsoft	User demand	OPEN
MFA method?	Security	Auth	TOTP, SMS	User needs	OPEN
Performance thresholds?	SLA	Infra, API	p95 < 500ms	Baseline	OPEN

Question	Why It Matters	Affected Areas	Known Options	Dependencies	Owner
Browser support?	UX	Frontend	Chrome, Firefox	Analytics	OPEN

34. Assumptions

Assumption	Why It Exists	Affected Area	Risk If Incorrect	Confirm/Invalidate
GitHub Actions for CI	Common practice	Infra	Wrong tooling	Check repo
PaaS hosting	MVP speed	Infra	Cost at scale	Monitor spend
Email verification deferred	MVP focus	Auth	Security gap	User feedback
OAuth/MFA post-MVP	Complexity	Auth	User demand	Survey

35. Recommendations

- Use Sentry for error tracking.
- Use k6 for load testing (Phase 4).
- Adopt JSON:API if need standardized responses (future).
- Consider RLS for tenant isolation at scale (Phase 3+).

36. Decision Conflicts

None identified. All docs align.

If found:

- Conflict.
- Sources.
- Current state.
- Impact.
- Required resolution.

37. Implementation Traceability

```
DEC-001 (Modular Monolith)
↓
architecture.md
↓
apps/api/src/modules/
↓
tests/integration/
```

↓
CI/CD (DEC-022)

38. Decision Review Triggers

Reconsider when:

- Scale exceeds assumptions.
- Security requirements change.
- Critical limitation appears.
- Product scope changes.
- Operational cost unacceptable.
- Dependency discontinued.
- Performance requirements change.
- Data model limitations significant.

39. Decision Ownership

Roles:

- Product owner: DEC-019 (MVP).
- Technical lead: DEC-001, DEC-003.
- Security lead: DEC-014, DEC-016.
- Architecture owner: DEC-001.

If unknown: Owner: OPEN DECISION.

40. Decision Review Process

```
Identify problem
↓
Check existing decisions
↓
Check source-of-truth documents
↓
Identify constraints
↓
Evaluate alternatives
↓
Assess impact
↓
Make decision
↓
Record decision
↓
```

Update affected documentation



Implement



Test

41. Decision Checklist for AI Agents

- ☐ Does an existing decision already answer this?
- ☐ Is the decision still active?
- ☐ Is it superseded?
- ☐ Does the requested change conflict?
- ☐ What source documents are affected?
- ☐ What are the alternatives?
- ☐ What are the consequences?
- ☐ What security implications exist?
- ☐ What database/API implications exist?
- ☐ What tests must change?
- ☐ Does the roadmap need updating?
- ☐ Does AGENTS.md need updating?
- ☐ Does a new DEC-ID need to be created?

42. Decision Checklist for Architectural Changes

- ☐ Problem clearly identified.
- ☐ Existing decision reviewed.
- ☐ Current implementation reviewed.
- ☐ Alternatives considered.
- ☐ Migration impact analyzed.
- ☐ Database/API/Auth/Security impact analyzed.
- ☐ Testing impact analyzed.
- ☐ Deployment impact analyzed.
- ☐ Backward compatibility considered.
- ☐ Rollback strategy considered.
- ☐ New decision documented.
- ☐ Old decision marked SUPERSEDED.
- ☐ Related documentation updated.

43. Decision Quality Criteria

Good decision:

- Clear, specific, evidence-based.
- Context-aware, honest about trade-offs.
- Reproducible, maintainable, traceable.
- Understandable by humans and AI.

Not:

- Marketing copy, generic tutorial.
- Personal opinion without evidence.
- Duplicate architecture doc.
- Arbitrary preferences.

44. Anti-Hallucination Rules

Do NOT invent:

- Historical decisions, dates, owners, reasons.
- Team discussions, alternatives supposedly considered.
- Technologies, architecture choices, business priorities.
- Security/product/performance constraints.

If rationale unknown: Rationale: Not documented.

If inferred from impl: Inferred from current implementation.

45. Existing Code vs Documented Decision

If discrepancy:

- Documented decision.
- Current implementation.
- Conflict.
- Impact.
- Required resolution.

Example:

- Doc: Modular monolith.
- Impl: Some microservice patterns.
- Conflict: Inconsistent.

- Impact: Confusion.
- Resolution: Align impl or update doc.

46. Decision Deduplication

Before creating:

- Search existing decisions.
- Search architecture docs.
- Search source-of-truth files.
- Extend/reference existing if covers topic.
- Only create new if necessary.

47. Decision Documentation Rule

Do not copy entire docs.

Instead:

- Record decision, rationale, consequences.
- Reference source doc.
- Reference related decisions.

48. Decision Reference

ID	Decision	Status	Category	Related Documents
DEC-001	Modular Monolith	CONFIRMED	Architecture	architecture.md
DEC-002	React + Vite	CONFIRMED	Technology	architecture.md
DEC-003	Node + Fastify	CONFIRMED	Technology	architecture.md
DEC-004	Prisma ORM	CONFIRMED	Technology	database.md
DEC-005	PostgreSQL	CONFIRMED	Database	database.md
DEC-006	UUID PKs	CONFIRMED	Database	database.md
DEC-007	Soft Delete	CONFIRMED	Database	database.md
DEC-008	Multi-Tenancy via org_id	CONFIRMED	Database	database.md , security.md
DEC-009	REST API	CONFIRMED	API	api-contract.md
DEC-010	Envelope Responses	CONFIRMED	API	api-contract.md
DEC-011	Cursor Pagination	CONFIRMED	API	api-contract.md
DEC-012	JWT Access + Refresh	CONFIRMED	Auth	auth.md

ID	Decision	Status	Category	Related Documents
DEC-013	httpOnly Refresh	CONFIRMED	Auth	auth.md
DEC-014	RBAC + Tenant Isolation	CONFIRMED	AuthZ	security.md
DEC-015	Zod Validation	CONFIRMED	Security	security.md
DEC-016	HMAC Webhooks	CONFIRMED	Security	security.md
DEC-017	Vitest + Playwright	CONFIRMED	Testing	testing.md
DEC-018	Ephemeral Test DB	CONFIRMED	Testing	testing.md
DEC-019	MVP Scope	CONFIRMED	Product	PRD.md, roadmap.md
DEC-020	Skeleton Loading	CONFIRMED	UX	testing.md
DEC-021	PaaS Hosting	CONFIRMED	Infra	roadmap.md
DEC-022	GitHub Actions CI/CD	CONFIRMED	Infra	roadmap.md
DEC-023	Monorepo with pnpm	CONFIRMED	Workflow	architecture.md
DEC-024	AI Agents Respect Decisions	CONFIRMED	AI	AGENTS.md

49. Current Accepted Decisions

Summary:

- Modular monolith (DEC-001).
- React + Vite + TS (DEC-002).
- Node + Fastify + TS (DEC-003).
- Prisma + PostgreSQL (DEC-004, DEC-005).
- UUID PKs, soft delete, org_id scoping (DEC-006, DEC-007, DEC-008).
- REST API, envelope, cursor pagination (DEC-009, DEC-010, DEC-011).
- JWT access + refresh, httpOnly cookies (DEC-012, DEC-013).
- RBAC + tenant isolation (DEC-014).
- Zod validation, HMAC webhooks (DEC-015, DEC-016).
- Vitest + Playwright, ephemeral test DB (DEC-017, DEC-018).
- MVP: Inventory + POS + WhatsApp + Payments (DEC-019).
- Skeleton loading, PaaS, GitHub Actions, monorepo (DEC-020, DEC-021, DEC-022, DEC-023).
- AI agents respect decisions (DEC-024).

50. Decision History

```
DEC-001 → Modular Monolith (CONFIRMED)
DEC-002 → React + Vite (CONFIRMED)
DEC-003 → Node + Fastify (CONFIRMED)
DEC-004 → Prisma ORM (CONFIRMED)
DEC-005 → PostgreSQL (CONFIRMED)
DEC-006 → UUID PKs (CONFIRMED)
DEC-007 → Soft Delete (CONFIRMED)
DEC-008 → Multi-Tenancy via `org_id` (CONFIRMED)
DEC-009 → REST API (CONFIRMED)
DEC-010 → Envelope Responses (CONFIRMED)
DEC-011 → Cursor Pagination (CONFIRMED)
DEC-012 → JWT Access + Refresh (CONFIRMED)
DEC-013 → httpOnly Refresh (CONFIRMED)
DEC-014 → RBAC + Tenant Isolation (CONFIRMED)
DEC-015 → Zod Validation (CONFIRMED)
DEC-016 → HMAC Webhooks (CONFIRMED)
DEC-017 → Vitest + Playwright (CONFIRMED)
DEC-018 → Ephemeral Test DB (CONFIRMED)
DEC-019 → MVP Scope (CONFIRMED)
DEC-020 → Skeleton Loading (CONFIRMED)
DEC-021 → PaaS Hosting (CONFIRMED)
DEC-022 → GitHub Actions CI/CD (CONFIRMED)
DEC-023 → Monorepo with pnpm (CONFIRMED)
DEC-024 → AI Agents Respect Decisions (CONFIRMED)
```

No superseded decisions yet.

51. Final Quality Review

- [x] Major existing decisions identified.
- [x] Based on actual project context.
- [x] Separated from assumptions/recommendations.
- [x] Open decisions identified.
- [x] Stable IDs.
- [x] No superseded decisions deleted.
- [x] Architecture/DB/API/Auth/AuthZ/Security/Testing/Product/Infra decisions recorded.
- [x] Trade-offs, alternatives, consequences documented.
- [x] Reconsideration triggers exist.
- [x] No conflicts identified.
- [x] AI rules explicit.
- [x] No fabricated rationale.
- [x] No unnecessary decisions.
- [x] Does not duplicate other docs.

52. Final Principle

AI coding agents must not make architectural decisions when an existing project decision governs that area.

Before significant technical choice:

Does a decision already exist?

↓

Is it still active?

↓

Does this task follow it?

↓

If yes → implement according to the decision.

↓

If no → identify the conflict.

↓

If the decision genuinely needs to change → create a new decision record and supersede

Preserve architectural consistency.

End of decisions.md